Nomenclature
The Google Go compiler and linkers are named as per Plan 9 with one letter for architecture (6 for amd64, 8 for x86, 5 for ARM) and the second is g for Go or l for linker

Mishmash
Go's syntax is inspired by C, and some its features by Pascal, Modula, Oberon, Newsqueak and Limbo

Functions in Go can return multiple values. A function which returns multiple values can be declared with a list of variables in parenthesis after the function as follows:

```
func f() (i int, j int);
v1, v2 = f();
```

• The FOR statements must be used without parenthesis, and the if statement can be used without them too. Any code which follows a for or if statement, should be wrapped in curly braces as follows:

```
if a < b
{
  //SOME CODE
}
for i = 0; i < 10; i++
{
  //SOME CODE
```

• Arrays are copied when passed to functions as they are first class values. To pass by reference, you use slices which are portions of an array complete with their own length and capacity.

Exploring OOP in Go

For someone who is quite used to classes derived from other classes derived from yet other classes, Interfaces might be a slightly jarring experience. Classes and inheritance allow for the creation of complicated hierarchies of relations between different elements. Inferfaces, on the other hand, encapsulate multiple types based on common properties. Interfaces are supported in many languages such as Java, ActionScript and C#. However in Go a type needn't declare what interfaces it implements.

An image button is a type of button which is a type of UI widget and so on, we start with the common base functionality and derive new elements from that.

In the above example we would code functionality which is common for any UI widget in a say the UIWidget class, and then derive that and add functionality for creating a Button. The Button class can then act as the base while we add functionality for an ImageButton.

Comparing to real life scenarios, this is the equivalent to creating a vehicle first, and then adding something to it to classify it as a car, and further add something to make it a specific model of a car. Although since you wouldn't be coding a car on your computer such analogies are useless for establishing the utility of the concept.

Interfaces work the other way around. Giving a real world example, as you go around the world looking you notice similarities in many of the things that you encounter, at which point you tend to group them in a collection. An observer who looks only at the night sky might call all twinkling lights stars, on further reflection he may call all twinkling lights in the sky stars, and after observing an aeroplane or two one might redefine them as stationary twinkling lights in the night sky.

You would call all portable computers laptops, and might then find it useful to classify some of them as netbooks based on certain properties that a subset exhibits. Such relationships are clear even with classes. However we now have many high-end mobiles / smartphones which could easily fall under the category of portable computers, as such smartphones and portable computers will have some shared items.

In the kind of interfaces provided in Go, you can aggregate multiple types into one interface based on their common properties. Looking at the example of mobiles: An interface for mobiles would require that the device be portable, that it allow voice communication and have a unique identifying phone number.

In programming languages such as Java, and C#, a type has to explicitly declare what all interfaces it implements. An interface then becomes, in a way, a promise that it will deliver at least the functionality defined in the interface declaration. In Go however we can define an interface with the functionality we'd want from it, and any type which provides the same will automatically implement it.

If we create an interface in Go for all types which are sortable, it would be something like the following:

```
type Sortable interface
{
    Len() int;
    Less(i, j int) bool;
    Swap(i, j int);
}
```

Now we create a function which does the actual sorting. In the function instead of specifying a particular type as a parameter, we use the interface instead, as follows:

```
func Sort(data Sortable)
{
    for i := 1; i < data.
Len(); i++
    {
        for j := i; j > 0 &&
data.Less(j, j-1); j--
        {
            data.Swap(j, j-1);
        }
    }
}
```

Now this function will work on any type which we define now or in the future with the three functions (Len, Less, and Swap) in it. While in other languages we would have had to manually specify which types implemented the interface Sortable; in Go any type we define now or in the future which implements these three functions (Len, Less, and Swap) will instantly be Sortable, and can be used by this function!

Conclusion

Google Go is still pretty much a language in incubation, as is many of their recent products and services, Google hopes that the community will help mold the direction of the product in the future. Google has unveiled the programming language and its source code early so that people can begin experimenting and playing with it already.

Google is already offering a choice of two compilers for the language, once which is a standalone compiler and linker, and another implementation which is built on GCC (GNU Compiler Collection).

Bad news for all the geeks out there using Windows though, the Go compiler is currently exclusively for Linux and Google has no plans to release a Windows version any time soon.

How far this language can go is something which will only start to become clear when the language reaches maturity, for now its only use remains in priding yourself in being one of the first few to know it.